

Compilation de machine abstraite vers Uxn

Timothée MENEUX

Stagiaire

Ecole Normale Supérieure

Rennes, France

timothee.meneux@ens-rennes.fr

Alan SCHMITT

Encadrant

Epicure, IRISA-INRIA

Rennes, France

alan.schmitt@inria.fr

Martin ANDRIEUX

Co-encadrant

Epicure, IRISA-INRIA

Rennes, France

martin.andrieux@inria.fr

Abstract—Alan SCHMITT et Martin ANDRIEUX travaillent sur un projet de traduction de description de sémantique en machine abstraite. Mon travail consistait alors à continuer la transformation en récupérant la machine abstraite et en la compilant vers le langage assembleur Uxn.

J’ai alors dans un premier temps écrit une petite machine abstraite étant un évaluateur d’expressions arithmétiques simples. J’ai ensuite appris à utiliser le langage assembleur Uxn. J’ai dû coder des fonctions de manipulation de la mémoire, un équivalent de *free* et *malloc*. Enfin, j’ai traduit à la main la petite machine abstraite en assembleur Uxn.

Dans un second temps, j’ai coder le compilateur en entier. Il s’appuie sur une chaîne de compilation avec plusieurs représentations internes et transformations sur ces représentations.

Index Terms—Machines Abstraites, Compilation, Uxn

I. INTRODUCTION

Contexte. Les machines abstraites prennent la forme d’interpréteurs récursifs terminaux et de premier ordre, sous forme d’un système de transitions [1]. Elles constituent le format de sémantique le plus précis et le plus bas niveau ; et sont donc particulièrement adaptées pour une implémentation directe en assembleur. Uxn est une architecture de jeu d’instructions minimale (elle ne contient que peu d’instructions) et open source.

Objet de recherche. Le projet consiste à développer un compilateur prenant en entrée une description de machine abstraite et produisant en sortie une implémentation de cette machine en assembleur Uxn.

Contributions. Le projet se déroulera en deux phases. Une première phase sera l’implémentation à la main d’une machine simple en Uxn. La deuxième phase consistera en la généralisation et l’automatisation de cette transformation, en utilisant nos outils de manipulations de sémantiques [2]. Si le temps le permet, la transformation pourra être prouvée correcte (sur papier).

Travaux des pairs. Ce projet s’intègre dans des travaux en cours sur la compilation générique de programmes [3] dont la sémantique est décrite en Skel, un langage haut niveau de description de sémantiques, et pour lequel un générateur de machines abstraites est déjà réalisé.

II. PREMIÈRE MACHINE ABSTRAITE

A. Syntaxe

On introduit notre machine abstraite cible, un évaluateur d’expressions arithmétiques simples.

$$\begin{aligned} \text{expr} ::= & \text{Int}(i) & i \in \mathbb{Z} \\ & | \text{Add}(\text{expr}, \text{expr}) \\ & | \text{Sub}(\text{expr}, \text{expr}) \end{aligned}$$

Figure 1: Syntaxe des expressions

$$\begin{aligned} \text{cont} ::= & \text{Id} \\ & | \text{Ar}_{\text{Add}}(\text{expr}, \text{cont}) \\ & | \text{Fn}_{\text{Add}}(\text{expr}, \text{cont}) \\ & | \text{Ar}_{\text{Sub}}(\text{expr}, \text{cont}) \\ & | \text{Fn}_{\text{Sub}}(\text{expr}, \text{cont}) \end{aligned}$$

Figure 2: Syntaxe des continuations

On notera $\mathcal{L}(\text{expr})$ et $\mathcal{L}(\text{cont})$ comme étant les langages reconnus par les grammaires *expr* et *cont*.

B. Sémantique concrète

Nos expressions admettent une sémantique concrète, celle de l’arithmétique. On évalue ces termes avec la fonction $\llbracket \cdot \rrbracket_{sc} : \mathcal{L}(\text{expr}) \rightarrow \mathbb{Z}$ définie comme suit :

$$\begin{aligned} \llbracket \text{Int}(i) \rrbracket_{sc} &= i \\ \llbracket \text{Add}(e_1, e_2) \rrbracket_{sc} &= \llbracket e_1 \rrbracket_{sc} + \llbracket e_2 \rrbracket_{sc} \\ \llbracket \text{Sub}(e_1, e_2) \rrbracket_{sc} &= \llbracket e_1 \rrbracket_{sc} - \llbracket e_2 \rrbracket_{sc} \end{aligned}$$

Figure 3: Sémantique concrète

C. Sémantique de la machine abstraite

Pour créer notre machine abstraite $\mathcal{M}^\sharp$, on la munie de trois états *eval*, *apply* et *res* ainsi que d’un système de transition $\rightarrow^\sharp$ étant une sémantique à petits pas.

$$\begin{aligned}
eval &:: \langle expr \mid cont \rangle_e \\
apply &:: \langle cont \mid expr \rangle_a \\
res &:: \langle i \rangle_r \mid \text{Fail} \quad i \in \mathbb{Z}
\end{aligned}$$

Figure 4: États de la machine abstraite

$$\begin{aligned}
\langle Int(i) \mid \kappa \rangle_e &\longrightarrow^\# \langle \kappa \mid i \rangle_a \quad i \in \mathbb{Z} \\
\langle Add(e_1, e_2) \mid \kappa \rangle_e &\longrightarrow^\# \langle e_1 \mid Ar_{Add}(e_2, \kappa) \rangle_e \\
\langle Sub(e_1, e_2) \mid \kappa \rangle_e &\longrightarrow^\# \langle e_1 \mid Ar_{Sub}(e_2, \kappa) \rangle_e
\end{aligned}$$

Figure 5: Transitions pour les états *eval*

$$\begin{aligned}
\langle Id \mid Int(i) \rangle_a &\longrightarrow^\# \langle i \rangle_r \\
\langle Ar_{Add}(e, \kappa) \mid Int(i) \rangle_a &\longrightarrow^\# \langle e \mid Fn_{Add}(Int(i), \kappa) \rangle_e \\
\langle Ar_{Sub}(e, \kappa) \mid Int(i) \rangle_a &\longrightarrow^\# \langle e \mid Fn_{Add}(Int(i), \kappa) \rangle_e \\
\langle Fn_{Add}(Int(i_1), \kappa) \mid Int(i_2) \rangle_a &\longrightarrow^\# \langle Int(i_1 + i_2) \mid \kappa \rangle_e \\
\langle Fn_{Sub}(Int(i_1), \kappa) \mid Int(i_2) \rangle_a &\longrightarrow^\# \langle Int(i_1 - i_2) \mid \kappa \rangle_e
\end{aligned}$$

Figure 6: Transitions pour les états *apply*

En Fig. 6, les opérations “+” et “−” représentent le vrai calcul opéré par la machine. De plus, les états *res* n’ont pas de successeur puisqu’ils symbolisent les états finaux de la machine.

On définit alors la clôture réflexive transitive de la relation de transition $\longrightarrow^\#$ par $\longrightarrow^*$. Pour deux états σ et σ' , on a $\sigma \longrightarrow^* \sigma'$ si et seulement si il existe une suite finie d’états $(\sigma_i)_{0 \leq i \leq n}$ telle que $\sigma_0 = \sigma$, $\sigma_n = \sigma'$ et pour tout $0 \leq i < n$, $\sigma_i \longrightarrow^\# \sigma_{i+1}$.

On pose donc la fonction d’évaluation associée à la machine abstraite $\llbracket \cdot \rrbracket^\# : \mathcal{L}(expr) \longrightarrow \mathbb{Z}$ comme étant le résultat de cette dernière sur l’entrée. Explicitement, on a $\llbracket e \rrbracket^\# = i$ si et seulement si $\langle e \mid Id \rangle_e \longrightarrow^* \langle i \rangle_r$.

III. AU NIVEAU MACHINE

A. Présentation rapide de Uxn

Uxn est un langage assembleur minimaliste qui s’exécute sur une machine (virtuelle) à pile normée Varvara. L’écosystème Uxn/Varvara permet d’exécuter la même application sur divers systèmes. Cet écosystème a par ailleurs été pensé pour opérer sur des systèmes léger, d’où son minimalisme.

En Uxn, on manipule une machine à pile simple ayant deux piles de 256 octets, une de travail et une de retour, ainsi qu’un espace mémoire adressable de 2^{16} octets (64ko).

Comme on manipule des piles, pour faire des calculs, on ajoute d’abord les opérandes sur la piles puis l’opérateur, l’opcode. Ainsi, en Uxn on écrit un code assembleur en notation polonaise inversée.

En Uxn, on dispose de 32 *opcodes* standards et 4 *opcodes* immédiats. Un *opcode* est un nom dont les trois premiers caractères figurent dans la table des *opcodes*, suivis d’une combinaison de “2”, “k” et “r”. Chaque *opcode* possède trois modes possibles, combinables :

- *short* (2) : manipule des *shorts* (2 octets) au lieu d’octets,
- *keep* (k) : ne consomme aucun élément,
- *return* (r) : manipule la pile de retour.

Par défaut, les *opcodes* consomment des octets de la pile de travail. Les *opcodes* immédiats admettent des “runes”, permettant de simplifier l’écriture du code. Par exemple, l’opcode immédiats “LIT” qui empile un octet sur la pile peut être remplacé par la rune “#”.

1	LIT 01	(ws: 01)	1	#09	(ws: 09)
2	#02	(ws: 02 01)	2	#01	(ws: 01 09)
3	ADD	(ws: 03)	3	#02	(ws: 02 01 09)
4	#09	(ws: 09 03)	4	ADD	(ws: 03 09)
5	MUL	(ws: 1b)	5	MUL	(ws: 1b)

Exemple 1: deux calculs de $(1 + 2) \times 9 = 27$ en Uxn, avec en commentaire l’état de la pile de travail.

Remarquons que dans l’exemple précédent (ex. 1), on peut déplacer des lignes sans que cela n’affecte le calcul.

B. Représentation machine des objets

Pour la représentation des objets, j’ai décidé de m’inspirer de la représentation à la OCaml. Ainsi, un objet est représenté par un pointeur vers une section mémoire contiguë qui peut être vue comme un triplet $\langle tag \mid size \mid data \rangle$ où :

- *tag* : indique le numéro du constructeur utilisé pour créer l’objet, codé sur un octet,
- *size* : indique le nombre de *shorts* réservés après, codé sur un octet,
- *data* : est l’espace utilisable en mémoire, un tableau de *size* shorts.

On remarque alors que la section mémoire réservée par un objet occupe $2 \times (size + 1)$ octets, soit $size + 1$ *shorts*. Ainsi, le champs *data* peut contenir au plus 255 shorts (car $size \leq 255 = 2^8 - 1$).

Il est intentionnel que les champs *tag* et *size* soient codés sur un octet chacun, ils sont contigus en mémoire et l’on ne travaillera qu’avec des *shorts*, ce qui est le cas lors de la concaténation de ces deux champs. Cette concaténation est alors appelée le *header*.

On appelle alors une case mémoire l’espace mémoire nécessaire pour stocker un *short* aligné.

C. Gestion mémoire

Afin de gérer la mémoire (de 64ko), on va équiper notre machine d'un équivalent de *malloc* et de *free*.

Le principe est simple. L'espace adressable est divisé en trois, une section *text*, une section *code* et une section *raw*. Les sections *text* et *code* contiennent le code assembleur chargé et les différentes variables d'environnement (qui ne peuvent pas avoir de valeur par défaut dans la section *text*).

Il nous reste alors toute la section *raw* qui peut être utilisée pour stocker dynamiquement des objets.

Lors d'un *malloc*, on va chercher la première case ayant un *tag* nul (ie le *tag* vaut 00) puis vérifier que les *size* cases suivantes ont aussi un *tag* nul. Si c'est le cas on écrit, sinon on reprend la recherche à partir de là. S'il n'y a plus de place, on renverra un pointeur nul (ie le pointeur 0000).

Pour effectuer un *free*, on a juste à écrire 00 sur le premier octet de chaque case de la section mémoire réservée par notre objet (*size* + 1 cases). Ainsi chaque case de la section mémoire sera alors considérée comme une case libre (*tag* 00).

IV. LE COMPILATEUR

A. La chaîne de compilation

L'objectif de ce compilateur est de recevoir en entrée le code *skel* d'une machine abstraite et de renvoyer le code Uxn compilé de cette machine abstraite.

Pour ce faire, on va définir plusieurs représentations sur lesquelles le compilateur travaillera. La première représentation est celle obtenue à partir du code *skel* via un AST (*Abstract Syntactic Tree*). La seconde est une représentation intermédiaire, que l'on nommera IR (*Intermediate Representation*), qui abstrait et simplifie le code reçu. Et enfin, la dernière représentation est la plus proche du langage d'Uxn.

Entre chaque représentation, on appliquera des transformations sur la représentation courante avant de la traduire et la transmettre à la représentation suivante.

Au début, on reçoit le code *skel* traduit pour notre AST. À la fin, une fois arrivé à la représentation en Uxn, on peut la compiler vers le langage assembleur d'Uxn.

On obtient alors la chaîne de compilation représentée en Fig. 7.

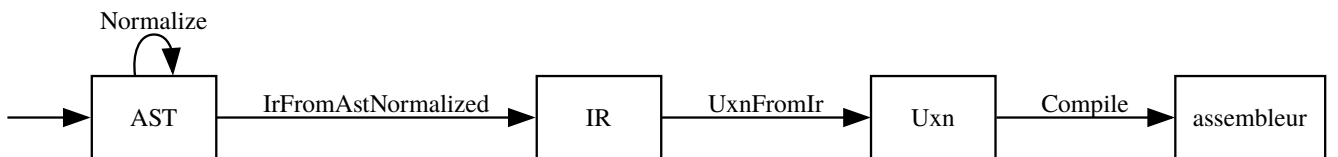


Figure 7: Chaîne de compilation

B. Première couche : l'AST

Pour créer l'AST, j'ai construit la grammaire que l'on pourrait décider d'utiliser pour *parser* les codes *skel* des machines abstraites. Une machine abstraite peut se représenter comme un tableau de transitions enrichi par des fonctions, internes et externes, et muni d'une fonction de transition principale.

Ainsi, une machine abstraite $\mathcal{M}^\#$ est représenté par un quintuplet $\langle e_T, e_F, T, F, \delta \rangle$ tel que :

- e_T est l'ensemble des types externes,
- e_F est l'ensemble des fonctions externes,
- T est l'ensemble des types internes,
- F est l'ensemble des fonctions internes,
- δ est la transition principale de $\mathcal{M}^\#$.

Les externes, types et fonctions, n'ont pas de définition explicite dans le code de $\mathcal{M}^\#$. Ces externes peuvent provenir d'Uxn (type : octet ou *short*, fonction : les *opcodes*) ou provenir d'une autre machine abstraite dont dépend alors $\mathcal{M}^\#$. Les externes sont alors juste représentés par une liste de noms associés à aucune définition.

Un type interne $t \in T$ est représenté par son nom et la liste de ses constructeurs.

Une fonction interne $f \in F$ est représentée par son nom, ses arguments, son typage et son corps. Le corps d'une fonction est un *statement*, défini en Fig. 10.

La transition principale δ est définie comme une fonction à ceci-près que son corp est directement *statement* **match** *x with* [...] (Fig. 10).

La grammaires définissant les *statement* (Fig. 10), dép des grammaires définissant les *expressions* (Fig. 8) et les *matching expressions* (Fig. 9).

Pour les définitions suivantes, x, o et f sont des chaînes de caractères, on notera $[type]$ une liste faite de mots de $\mathcal{L}(type)$ et on notera $\rightarrow (type_1, type_2)$ un element de $\mathcal{L}(type_1) \times \mathcal{L}(type_2)$.

```

expr ::= Var(x)
      | Tuple([expr])
      | Object(o, [expr])
      | FunCall(f, [expr])

```

Figure 8: Grammaire des *expressions* de l'AST

```

matchExpr ::= MatchVar (x)
           | MatchTuple([matchExpr])
           | MatchObject(o, [matchExpr])

```

Figure 9: Grammaire des *matching expressions* de l'AST

```

statement ::= expr
           | let x = expr in statement
           | match x with
             [matchExpr → statement]

```

Figure 10: Grammaire des *statements* de l'AST

C. Normalisation de l'AST

Dans l'état actuel, il n'est pas pratique de traduire directement le code vers Uxn. En effet, le problème vient des *matching expressions*. Comme ces expressions sont récursives, on peut être amené à faire un match récursif, ce qui n'est pas pratique à coder en Uxn. Le problème est qu'on devrait progressivement charger le premier *short* de l'objet afin de récupérer son *header*, ce qu'on utilisera pour le matching.

L'objectif est alors de transformer les *statements* pour que les *matching expressions* n'ai plus qu'une hauteur au plus égale à un. Où la hauteur est vu classiquement sur les arbres, avec les `MatchVar(x)` qui sont les feuilles de hauteur 0.

```

1 match a with
2 | Cons1 (Cons2 b, c) → s1
3 | Cons1 (Cons3 (d, e), f) → s2
4 | ...

```

↯ Normalize ↯

```

1 match a with
2 | Cons1 (x, y) → (
3   match x with
4   | Cons2 b → s1{c := y}
5   | Cons3 (d,e) → s2{f := y}
6   ...
7 )
8 | ...

```

Exemple 2: normalisation de match (en OCaml).

En prenant l'exemple précédent (ex. 2), le *statement* initial est normalisé en le *statement* final. On remarque

alors qu'on a besoin d'introduire des variables fraîches et de fusionner des variables sous un même nom, grâce aussi à des variables fraîches. Il faut alors substituer dans le *statement* résultant du cas du match l'ancienne variable par la nouvelle.

Ainsi, cette normalisation va être découpée en deux phases : une première de décurrification puis une seconde de fusion.

TABLE I: définition de la fonction *decurrify*

m_e	$decurrify(m_e, s)$
<code>MatchVar(x)</code>	(m_e, s)
<code>MatchTuple(l)</code>	$(MatchTuple(l'), s')$ avec : $(l', s') = \text{fold_right}(\text{rebuild}, l, ([], s))$
<code>MatchObject(o, l)</code>	$(MatchObject(o, l'), s')$ avec : $(l', s') = \text{fold_right}(\text{rebuild}, l, ([], s))$

TABLE II: définition de la fonction *rebuild*

m_e	$\text{rebuild}(m_e, (l, s))$
<code>MatchVar(x)</code>	$(m_e :: l, s)$
<code>MatchTuple(l')</code>	$(MatchVar(x) :: l,$ $\text{match } x \text{ with } [m'_e \rightarrow s'])$ avec : $(m'_e, s') = \text{decurrify}(m_e, s)$ x une variable fraîche
<code>MatchObject(o, l')</code>	

Pour effectuer la première phase de décurrification, on fait appel à deux fonctions mutuellement récursives : *decurrify* et *rebuild*. La fonction *decurrify* remplace les arguments des *matching expressions* par des arguments qui ne sont que sous la forme `MatchVar(x)` obtenus à partir de la fonction *rebuild*. Cette deuxième fonction prend en entrée une *matching expression*, la liste des arguments déjà reconstruits et le corps du *match* puis renvoie la nouvelle liste d'arguments, agrandie de un élément, et le corps du *match* potentiellement étendu par un autre *match* qui aura été passé dans *decurrify*.

À présent, il va falloir fusionner les cas du *match*. En effet, notre transformation introduit des cas de *matching* sur un même constructeur qui n'opère alors plus que sur des variables libres. Ainsi, seul le premier cas de chaque constructeur peut être visité. L'algorithme de fusion est assez intuitif, on renomme les arguments de constructeurs pour que tous les arguments en même position aient le même nom puis on substitue dans le corps du *match*. Enfin, on fusionne les branches et on itère récursivement.

On obtient alors une chaîne de normalisation, comme représentée en ex. 3, qui nous permet d'avoir un pro-

gramme dont les *matching expressions* ont bien une hauteur au plus égale à un.

```

1 match a with
2 | Cons1 (Cons2 b, c) → s1
3 | Cons1 (Cons3 (d, e), f) → s2
4 | ...

```

↳ decurrify ↳

```

1 match a with
2 | Cons1 (x1, c) → (
3     match x1 with
4     | Cons2 b → s1
5 )
6 | Cons1 (x2, f) → (
7     match x2 with
8     | Cons3 (d,e) → s2
9 )
10 | ...

```

↳ merge ↳

```

1 match a with
2 | Cons1 (x1, y) → (
3     match x1 with
4     | Cons2 b → s1
5     | Cons3 (d,e) → s2{x2 := x1, f := c}
6     ...
7 )
8 | ...

```

Exemple 3: chaîne de normalisation de match (en OCaml).

D. Seconde couche : l'IR

Cette représentation intermédiaire sert à simplifier la représentation précédente, tout en gardant la même expressivité, afin de simplifier la traduction vers la couche suivante, la représentation d'un programme Uxn.

Ainsi, la traduction de l'AST vers l'IR est assez immédiate. Il y a quelques renommage en cours de route, comme les arguments d'un objets dans les *matching expressions* qui sont alors renommée en le nom de

la variable auquel on ajoute en suffixe le numéro sa position dans les arguments.

L'intérêt principale de cette représentation est la traduction qui est faite pour les types. À présent, un type n'est plus que définit par la liste de ses constructeurs associé au nombre d'arguments qu'il requiert.

Dans le même temps, les tuples deviennent des objets normaux. Le constructeur est alors "Tuple" et est associé au nombre d'éléments que contient le tuple.

E. Dernière couche : Uxn

Dans cette représentation, on intègre tout ce qui sera nécessaire à la compilation vers Uxn.

Ainsi, on intègre à présent la pile dans la définition, afin de simuler l'état de cette dernière au fil de l'exécution. De plus, comme on n'a plus exactement des variables, on créer un type *var* qui est soit une variable qui est alors stockée dans la pile, soit un accès mémoire, où l'on empile alors le résultat.

Comme petite modification, maintenant on distingue les appels de fonction des *opcodes* d'Uxn.

La plus grosse transformation s'opère encore sur les *matches*. À présent, chaque corps de cas d'un *match* est un appel à une fonction qui elle contient le code de l'action à effectuer.

```

1 match a with
2 | Cons (...) → s
3 | ...

```

↳

```

1 let f() = s'

```

```

1 match a with
2 | Cons → f()
3 | ...

```

Exemple 4: transformation de match (en OCaml), avec s' étant le résultat de la transformation de s .

Aussi, tous les noeuds de notre arbre syntaxique sont annoté par l'état de la pile avant leur exécution et l'état théorique après leur exécution.

F. Export final

Il nous faut d'abord exporter les types. Ils seront représenté comme un macro en Uxn. Une macro en Uxn est un mot qui est associé à un suite d'instructions et qui remplace statiquement ses usages à l'assemblage. Avec le travail que l'on a effectué jusqu'à présent, nos types et nos constructeurs correspondent presque aux *headers* des objets représenté en mémoire, cf Section III.B. Il

faut juste récupérer l'index du constructeur dans la liste de son type ainsi que le nombre d'argument et l'on a notre macro.

```
configuration = Constructors[Eval(2)
                               ; Apply(2)
                               ; Stop(1)
                               ; Main(1)]
```

↯

```
1 ( configuration )
2 %Eval { #0102 }
3 %Apply { #0202 }
4 %Stop { #0301 }
5 %Main { #0401 }
```

Exemple 5: Compilation d'un type

Comme on le voit en ex. 5, le constructeur Eval est le premier de la liste, donc il commence par 01, puis il prend 2 arguments, d'où 0102.

Ensuite, il faut compiler les expressions (valeur, création d'objets et appel de fonctions). Cette phase là se fait assez facilement avec les *opcodes* d'Uxn tout en manipulant les piles. Deux cas restent néanmoins non-triviaux, le cas des valeurs (variables ou accès mémoire).

Une valeur est soit une variable qui est déjà présente dans la pile de travail soit un accès mémoire. Pour récupérer un variable dans la pile de travail, on dépile la pile de travail dans la pile de retour (opération permise en Uxn) jusqu'à arriver à notre variable, qu'on duplique (opération permise en Uxn) puis on fait l'inverse, on dépile la pile de retour dans la pile de travail sous notre variable, et l'on s'arrête une fois que la pile de retour a retrouvé son état initial. Si c'est un accès mémoire, le $i^{\text{ème}}$ élément d'un objet, on récupère le pointeur dans la pile auquel on ajoute $2 \times (i + 1)$. On ajoute un car $i = 0$ correspond au *header* de notre objet et on multiplie par 2 car on peut accéder à chaque octet de la mémoire, or l'on a décidé de ne considérer que des *shorts*.

Ensuite, il faut compiler les *statements*. Là aussi c'est assez naturel, seul le cas des *matches* est intéressant à détailler.

Dans notre représentation actuelle, un *match* n'est plus qu'une valeur qui admet une liste de cas représentés seulement par le nom du constructeur et où pour chaque cas on saute vers la bonne fonction. Ainsi, on va comparer le constructeurs avec le *header* de notre objet, si c'est les mêmes on saute vers la fonction, sinon, on passe au cas suivant.

```
1 @step
2   LDA2k Apply EQU2 ;step<Apply> JCN2
3   LDA2k Eval EQU2 ;step<Eval> JCN2
4   LDA2k Main EQU2 ;step<Main> JCN2
5   LDA2k Stop EQU2 ;step<Stop> JCN2
```

Exemple 6: Compilation d'un match sur un type

Détaillons l'exemple précédent (ex. 6), qui est un match sur le type présenté avant (ex. 5). On est dans la fonction *step*, symbolisé par le "@" devant le nom, et l'on fait un match sur l'objet que l'on reçoit. Comme l'objet que l'on reçoit est un pointeur vers la mémoire comme vu en Section III.B, sa première case mémoire est le *header*. En faisant LDA2k, on empile le *header* en conservant l'objet en dessous. Ensuite on empile le *header* des objets construits par *Apply* puis on les compare, on consomme les 2 *headers* et il ne reste plus qu'un octet représentant un booléen. Finalement, on empile l'adresse de la fonction qui gère le cas *Apply* et on saute à cette adresse si le booléen est vrai, le booléen et l'adresse sont consommés. Sinon, on passe à la ligne suivante et ça marche pareil.

V. CE QU'IL RESTE À FAIRE

Je dois rajouter le *free* et je dois modifier le *malloc* pour qu'il recherche vraiment de l'espace libre car pour le moment c'est une simple pile avec un *push*.

Je dois aussi corriger les quelques bug qui restent.

REFERENCES

- [1] M. Ager, "A Functional Correspondence Between Evaluators and Abstract Machines," 2003, doi: 10.1145/888251.888254.
- [2] A. SCHMITT, "Skeletons." [Online]. Available: <https://skeletons.inria.fr/>
- [3] S. Diehl, P. Hartel, and P. Sestoft, "Abstract machines for programming language implementation," *Future Generation Computer Systems*, vol. 16, no. 7, pp. 739–751, 2000, doi: [https://doi.org/10.1016/S0167-739X\(99\)00088-6](https://doi.org/10.1016/S0167-739X(99)00088-6).